

✓ Smart Grocery Cart Assistant

Built on top of the JioMart Retail Product Catalog ([Kaggle](#)).

A Gradio-based AI assistant that recommends a weekly grocery shopping cart using Retrieval-Augmented Generation (RAG) over a structured product catalog, with explainable, structured output rendered as a visual shopping cart.

This notebook implements:

- Part 1 — Core RAG assistant (free-form dietary preferences → structured cart)
- Part 2 — Visual cart rendering (`gr.Gallery`)
- Part 3 — Advanced LLM settings (model, temperature, **top_p**, collapsible UI)
- Part 4 — Advanced retrieval (**Top-K**, chunk size/overlap, MMR / hybrid search)
- Part 4 — Advanced user features (**budget constraint, 7-day diet plan + ingredient list**)

```
# Setup
import gradio as gr
import pandas as pd
import os
import getpass

from dotenv import load_dotenv

from langchain_chroma import Chroma
from langchain_ollama import OllamaEmbeddings
from langchain_community.document_loaders import CSVLoader
from langchain.chat_models import init_chat_model
from langchain_core.documents import Document
from langchain_core.prompts import ChatPromptTemplate
from langchain_core.output_parsers import PydanticOutputParser
from langchain_core.exceptions import OutputParserException
from langchain_text_splitters import RecursiveCharacterTextSplitter
from langchain.retrievers import EnsembleRetriever
from langchain_community.retrievers import BM25Retriever

from pydantic import BaseModel, Field
from typing import List, Optional
```

```
# Load environment variables and API keys
load_dotenv(override=True)

if "GROQ_API_KEY" not in os.environ:
    os.environ["GROQ_API_KEY"] = getpass.getpass("GROQ API KEY: ")
```

✓ Data Loading & Preprocessing

Load the JioMart product catalog and normalize each row into a LangChain `Document`.

```
# Load and preprocess data
loader = CSVLoader(file_path="jiomart_products_database.csv", source_column="title")
documents_raw = loader.load()
```

```
# Convert page_content string to dict and build metadata
documents = []
for doc in documents_raw:
    try:
        row_data = dict(
            line.split(":", 1) for line in doc.page_content.split("\n") if ":" in line
        )
        row_data = {k.strip(): v.strip() for k, v in row_data.items()}

        page_text = f"Name: {row_data.get('title', '')} | Sub-type: {row_data.get('subType', '')} | Type: {row_data.get('type', '')} | Price: {row_data.get('discountedPrice', 0)} | Image: "
        metadata = {
            "category": row_data.get("type", ""),
            "sub_category": row_data.get("subType", ""),
            "title": row_data.get("title", ""),
            "price": row_data.get("discountedPrice", "0"),
            "image_url": row_data.get("filename", "")
        }

        documents.append(Document(page_content=page_text, metadata=metadata))

    except Exception as e:
        print("Skipping row due to error:", e)

print(f"Loaded {len(documents)} product documents")
```

Loaded 5672 product documents

documents[:3]

```
[Document(metadata={'category': 'Staples', 'sub_category': 'Atta, Flours and Sooji', 'title': 'Besan 1 kg', 'price': '74', 'image_url':
'https://www.jiomart.com/images/product/150x150/491349649/besan-1-kg-0-20210521.jpg'}, page_content='Name: Besan 1 kg | Sub-type: Atta, Flours and Sooji | Type: Staples | Price: 74 |
Image: https://www.jiomart.com/images/product/150x150/491349649/besan-1-kg-0-20210521.jpg'),
Document(metadata={'category': 'Staples', 'sub_category': 'Atta, Flours and Sooji', 'title': 'Besan 500 g', 'price': '37', 'image_url':
'https://www.jiomart.com/images/product/150x150/491349648/besan-500-g-0-20210521.jpg'}, page_content='Name: Besan 500 g | Sub-type: Atta, Flours and Sooji | Type: Staples | Price: 37 |
Image: https://www.jiomart.com/images/product/150x150/491349648/besan-500-g-0-20210521.jpg'),
Document(metadata={'category': 'Staples', 'sub_category': 'Atta, Flours and Sooji', 'title': 'Maida 1 kg', 'price': '32', 'image_url':
'https://www.jiomart.com/images/product/150x150/491349662/maida-1-kg-0-20210521.jpg'}, page_content='Name: Maida 1 kg | Sub-type: Atta, Flours and Sooji | Type: Staples | Price: 32 |
Image: https://www.jiomart.com/images/product/150x150/491349662/maida-1-kg-0-20210521.jpg')]
```

✓ Chunking (Advanced — Developer Setting)

Product entries are short, so chunking mainly matters if `title`/description fields are long. We expose **chunk size** and **chunk overlap** as configurable parameters (wired to the UI later) and re-chunk the corpus through `RecursiveCharacterTextSplitter`. For short catalog

rows most documents will remain a single chunk, which is expected — splitting still guards against any unusually long product descriptions.

```
def chunk_documents(docs, chunk_size=500, chunk_overlap=50):
    """Re-chunk documents using the given chunk size / overlap."""
    splitter = RecursiveCharacterTextSplitter(
        chunk_size=chunk_size,
        chunk_overlap=chunk_overlap,
        separators=["\n\n", "\n", " | ", " ", " ", ""]
    )
    return splitter.split_documents(docs)

# Default chunking pass (re-applied later per UI settings when the index is rebuilt)
DEFAULT_CHUNK_SIZE = 500
DEFAULT_CHUNK_OVERLAP = 50

chunked_documents = chunk_documents(documents, DEFAULT_CHUNK_SIZE, DEFAULT_CHUNK_OVERLAP)
print(f"{len(documents)} source docs -> {len(chunked_documents)} chunks (size={DEFAULT_CHUNK_SIZE}, overlap={DEFAULT_CHUNK_OVERLAP})")
```

```
5672 source docs -> 5675 chunks (size=500, overlap=50)
```

✓ Embeddings & Vector Store

Embeddings run locally via **Ollama** (`nomic-embed-text`). Make sure Ollama is running locally and the model is pulled:

```
ollama pull nomic-embed-text
ollama serve
```

```
# Create embeddings (local Ollama)
embeddings_model = OllamaEmbeddings(
    model="nomic-embed-text"
)
```

```
INDEX_LIMIT = 500

documents_for_chroma = chunked_documents if INDEX_LIMIT is None else chunked_documents[:INDEX_LIMIT]

vectorstore = Chroma.from_documents(
    documents=documents_for_chroma,
    embedding=embeddings_model,
    collection_name="jiomart_products"
)

print(f"Indexed {len(documents_for_chroma)} chunks into Chroma")
```

```
Indexed 500 chunks into Chroma
```

Retriever — Top-K, MMR & Hybrid Search (Advanced — Developer Setting)

Three retrieval strategies are supported, selectable from the UI:

- **Similarity** — plain vector similarity search (top-K nearest neighbors)
- **MMR** — Maximal Marginal Relevance, trades off relevance vs. diversity to avoid returning near-duplicate products
- **Hybrid** — combines dense vector search (Chroma) with sparse keyword search (BM25) via an `EnsembleRetriever`, useful when users mention exact product names or brand terms that embeddings alone may under-rank

```
# BM25 retriever for hybrid search (sparse / keyword-based)
bm25_retriever = BM25Retriever.from_documents(documents_for_chroma)
bm25_retriever.k = 5

def build_retriever(top_k=5, search_type="similarity", mmr_lambda=0.5, hybrid_weight=0.5):
    """
    Build a retriever based on the selected strategy.

    top_k      : number of products to retrieve
    search_type : "similarity" | "mmr" | "hybrid"
    mmr_lambda  : MMR diversity parameter (0 = max diversity, 1 = max relevance)
    hybrid_weight : weight given to the dense (vector) retriever in hybrid mode;
                  the BM25 retriever gets (1 - hybrid_weight)

    """
    bm25_retriever.k = top_k

    if search_type == "mmr":
        return vectorstore.as_retriever(
            search_type="mmr",
            search_kwargs={"k": top_k, "lambda_mult": mmr_lambda, "fetch_k": max(top_k * 4, 20)}
        )

    if search_type == "hybrid":
        dense_retriever = vectorstore.as_retriever(search_kwargs={"k": top_k})
        return EnsembleRetriever(
            retrievers=[dense_retriever, bm25_retriever],
            weights=[hybrid_weight, 1 - hybrid_weight]
        )

    # default: plain similarity
    return vectorstore.as_retriever(search_kwargs={"k": top_k})

# Default retriever (rebuilt per-request using UI settings inside generate_cart)
retriever = build_retriever(top_k=5, search_type="similarity")
```

Structured Output Schema & Prompt

`GroceryItem` / `GroceryOutput` define the structured cart. The prompt now also accepts an optional **budget** constraint and an optional **diet-plan** request, both wired to the Advanced user-facing settings.

```

class GroceryItem(BaseModel):
    item_name: str
    price: float
    quantity: int
    image_url: str

class DayPlan(BaseModel):
    day: str = Field(description="Day of the week, e.g. 'Monday'")
    meals: str = Field(description="Short description of meals planned for the day, referencing cart ingredients")

class GroceryOutput(BaseModel):
    reasoning: str = Field(description="Concise explanation of why these products were selected")
    items: List[GroceryItem]
    total_cost: Optional[float] = Field(default=None, description="Sum of quantity * price across all items")
    diet_plan: Optional[List[DayPlan]] = Field(
        default=None,
        description="7-day meal plan using the selected ingredients, only populated if the user requested a diet plan"
    )

# Initialize parser
parser = PydanticOutputParser(pydantic_object=GroceryOutput)

# Prompt template – now budget- and diet-plan-aware
prompt = ChatPromptTemplate.from_template(
    """
You are a smart grocery shopping assistant.

Your task is to create a grocery shopping cart based on
the user's preferences.

USER PREFERENCES:
{preferences}

BUDGET CONSTRAINT:
{budget_instruction}

DIET PLAN REQUEST:
{diet_plan_instruction}

AVAILABLE PRODUCTS:
{context}

IMPORTANT RULES:

1. Recommend products ONLY from the available products
   provided in the context.

2. Do NOT invent products.

3. Do NOT recommend products that the user explicitly
   excluded.

```

4. Use the exact product name from the context.
5. Use the exact price from the context.
6. Use the image URL from the context.
7. Select a reasonable quantity based on the user's request.
8. If the user asks for a specific type of product, prioritize products matching that requirement.
9. If a budget constraint is given, keep the total cost (sum of quantity * price across all items) at or below the budget. Prioritize higher-value / higher-relevance items first and drop lower-priority items if needed to stay within budget. Populate total_cost accordingly.
10. If a diet plan was requested, populate diet_plan with a 7-day (Monday-Sunday) plan that only uses ingredients from the selected cart items. Keep each day's meals field brief (1-2 sentences). If no diet plan was requested, leave diet_plan empty.
11. Keep the reasoning concise and explain why the selected products match the user's preferences.
12. Return ONLY the requested structured output.

```
{format_instructions}
"""
)
```

▼ Model Selector (Groq)

Three Groq-hosted models, chosen after checking current leaderboard performance and throughput trade-offs. Embeddings stay local via Ollama; generation stays on Groq as decided for this build.

```
model_name_map = {
    "GPT-OSS-120B": "openai/gpt-oss-120b",
    "Llama 3.3 70B": "llama-3.3-70b-versatile",
    "Llama 4 Scout": "meta-llama/llama-4-scout-17b-16e-instruct"
}

model_choices = [
    "GPT-OSS-120B (Groq)",
    "Llama 3.3 70B (Groq)",
    "Llama 4 Scout (Groq)"
]

def get_llm(model_choice, temperature, top_p):
    """Initialize the selected Groq chat model with temperature + top_p."""
```

```

# Remove "(Groq)" suffix from UI label
model_key = model_choice.replace(" (Groq)", "")
model_name = model_name_map[model_key]

llm = init_chat_model(
    model=model_name,
    model_provider="groq",
    temperature=temperature,
    top_p=top_p
)

return llm

```

v RAG Pipeline

End-to-end: retrieve (with configurable strategy/top-K) → format context → build prompt (with budget + diet-plan instructions) → call LLM (with temperature + top_p) → parse structured output.

```

def generate_cart(user_input):
    """
    user_input keys:
    preferences      : str  (free-form dietary needs)
    model_choice     : str
    temperature      : float
    top_p            : float
    top_k            : int  (retrieval top-K)
    search_type      : str  ("similarity" | "mmr" | "hybrid")
    mmr_lambda       : float
    hybrid_weight    : float
    budget           : float or None
    want_diet_plan   : bool
    """
    try:
        # -----
        # STEP 1: Build retriever per current settings & retrieve
        # -----
        preferences = user_input["preferences"]

        active_retriever = build_retriever(
            top_k=user_input.get("top_k", 5),
            search_type=user_input.get("search_type", "similarity"),
            mmr_lambda=user_input.get("mmr_lambda", 0.5),
            hybrid_weight=user_input.get("hybrid_weight", 0.5)
        )

        context_docs = active_retriever.invoke(preferences)

        # -----
        # STEP 2: Convert retrieved documents to text
        # -----
        relevant_text = "\n\n".join(doc.page_content for doc in context_docs)

```

```

if not relevant_text.strip():
    return "No matching products found for your preferences. Try rephrasing.", None

# -----
# STEP 3: Get selected LLM
# -----
llm = get_llm(
    user_input["model_choice"],
    user_input["temperature"],
    user_input["top_p"]
)

# -----
# STEP 4: Format prompt (budget + diet plan instructions)
# -----
budget = user_input.get("budget")
budget_instruction = (
    f"Keep the total cart cost at or below Rs. {budget:.2f}."
    if budget else "No budget constraint specified."
)

diet_plan_instruction = (
    "The user wants a 7-day diet plan using the selected ingredients. Populate diet_plan."
    if user_input.get("want_diet_plan") else
    "No diet plan requested. Leave diet_plan empty."
)

formatted_prompt = prompt.format(
    preferences=preferences,
    context=relevant_text,
    budget_instruction=budget_instruction,
    diet_plan_instruction=diet_plan_instruction,
    format_instructions=parser.get_format_instructions()
)

# -----
# STEP 5: Call LLM
# -----
output = llm.invoke(formatted_prompt)

# -----
# STEP 6: Parse output
# -----
try:
    result = parser.parse(output.content)
except OutputParserException:
    return "Could not parse output. Try again or adjust temperature.", None

# Fallback: compute total_cost if the model didn't fill it in
if result.total_cost is None:
    result.total_cost = sum(item.quantity * item.price for item in result.items)

return result

except Exception as e:
    print("Error in generate_cart:", e)

```

```
return f"Error: {str(e)}", None
```

✓ Gradio UI

- **Basic:** free-form preferences box, budget field, diet-plan checkbox
- **Advanced (collapsible `gr.Accordion`):**
 - LLM settings: model, temperature, **top_p**
 - Retrieval settings: search type (similarity / MMR / hybrid), **top_k**, MMR lambda, hybrid weight, chunk size, chunk overlap
- **Output:** reasoning textbox, shopping cart gallery, optional 7-day diet plan table, total cost

```
def gradio_interface(
    preferences, budget, want_diet_plan,
    model_choice, temperature, top_p,
    search_type, top_k, mmr_lambda, hybrid_weight,
    chunk_size, chunk_overlap
):
    global chunked_documents, documents_for_chroma, vectorstore, bm25_retriever

    # Rebuild the index only if chunking parameters changed from what's currently indexed
    if chunk_size != DEFAULT_CHUNK_SIZE_STATE["size"] or chunk_overlap != DEFAULT_CHUNK_SIZE_STATE["overlap"]:
        chunked_documents = chunk_documents(documents, chunk_size, chunk_overlap)
        documents_for_chroma = chunked_documents if INDEX_LIMIT is None else chunked_documents[:INDEX_LIMIT]
        vectorstore = Chroma.from_documents(
            documents=documents_for_chroma,
            embedding=embeddings_model,
            collection_name="jiomart_products"
        )
        bm25_retriever = BM25Retriever.from_documents(documents_for_chroma)
        DEFAULT_CHUNK_SIZE_STATE["size"] = chunk_size
        DEFAULT_CHUNK_SIZE_STATE["overlap"] = chunk_overlap

    user_input = {
        "preferences": preferences,
        "model_choice": model_choice,
        "temperature": temperature,
        "top_p": top_p,
        "top_k": int(top_k),
        "search_type": search_type,
        "mmr_lambda": mmr_lambda,
        "hybrid_weight": hybrid_weight,
        "budget": budget if budget and budget > 0 else None,
        "want_diet_plan": want_diet_plan,
    }

    result = generate_cart(user_input)

    if not result or isinstance(result, str):
        return result, None, "", ""

    if isinstance(result, tuple):
        explanation, = result
```

```

        return explanation, None, "", ""

# Shopping cart gallery: [(image_url, caption), ...]
gallery_items = [
    (item.image_url, f"{item.item_name}\nQty.{item.quantity}\nRs.{item.quantity * item.price:.2f}")
    for item in result.items
]

total_cost_text = f"""Total: Rs. {result.total_cost:.2f}""" if result.total_cost is not None else ""

diet_plan_text = ""
if result.diet_plan:
    lines = [f"""{day.day}*: {day.meals}""" for day in result.diet_plan]
    diet_plan_text = "### 7-Day Diet Plan\n\n" + "\n\n".join(lines)

return result.reasoning, gallery_items, total_cost_text, diet_plan_text

# Tracks the chunking config currently reflected in the live index, so we only
# rebuild Chroma/BM25 when the user actually changes chunk size/overlap.
DEFAULT_CHUNK_SIZE_STATE = {"size": DEFAULT_CHUNK_SIZE, "overlap": DEFAULT_CHUNK_OVERLAP}

with gr.Blocks(title="Smart Grocery Cart Assistant") as demo:
    gr.Markdown("# Smart Grocery Cart Assistant")
    gr.Markdown("Get a product list tailored to your dietary preferences.")

    with gr.Row():
        with gr.Column(scale=2):
            preferences_input = gr.Textbox(
                label="Describe your grocery needs",
                placeholder="e.g. 'high protein, no besan or curd'",
                lines=2
            )
        with gr.Row():
            budget_input = gr.Number(label="Budget (Rs., optional)", value=0, minimum=0)
            diet_plan_checkbox = gr.Checkbox(label="Include 7-day diet plan", value=False)

    submit_btn = gr.Button("Generate Cart", variant="primary")

    with gr.Accordion("LLM Settings (Advanced)", open=False):
        model_dropdown = gr.Dropdown(label="Model", choices=model_choices, value=model_choices[0])
        temperature_slider = gr.Slider(minimum=0.0, maximum=1.5, value=0.7, step=0.1, label="Temperature")
        top_p_slider = gr.Slider(minimum=0.0, maximum=1.0, value=1.0, step=0.05, label="Top P")

    with gr.Accordion("Retrieval Settings (Advanced)", open=False):
        search_type_radio = gr.Radio(
            choices=["similarity", "mmr", "hybrid"],
            value="similarity",
            label="Search Strategy"
        )
        top_k_slider = gr.Slider(minimum=1, maximum=20, value=5, step=1, label="Top K (products retrieved)")
        mmr_lambda_slider = gr.Slider(minimum=0.0, maximum=1.0, value=0.5, step=0.05, label="MMR Lambda (relevance vs diversity)")
        hybrid_weight_slider = gr.Slider(minimum=0.0, maximum=1.0, value=0.5, step=0.05, label="Hybrid Weight (vector vs keyword)")
        chunk_size_slider = gr.Slider(minimum=100, maximum=2000, value=DEFAULT_CHUNK_SIZE, step=50, label="Chunk Size")
        chunk_overlap_slider = gr.Slider(minimum=0, maximum=500, value=DEFAULT_CHUNK_OVERLAP, step=10, label="Chunk Overlap")

```

```

with gr.Column(scale=3):
    reasoning_output = gr.Textbox(label="Considerations", lines=4)
    total_cost_output = gr.Markdown()
    cart_gallery = gr.Gallery(label="Shopping Cart", columns=3, height="auto")
    diet_plan_output = gr.Markdown()

submit_btn.click(
    fn=gradio_interface,
    inputs=[
        preferences_input, budget_input, diet_plan_checkbox,
        model_dropdown, temperature_slider, top_p_slider,
        search_type_radio, top_k_slider, mmr_lambda_slider, hybrid_weight_slider,
        chunk_size_slider, chunk_overlap_slider
    ],
    outputs=[reasoning_output, cart_gallery, total_cost_output, diet_plan_output]
)

# Launch UI
if __name__ == "__main__":
    demo.launch()

```

* Running on local URL: <http://127.0.0.1:7869>

* To create a public link, set `share=True` in `launch()`.



```

d:\JioInstitute\Term 4\GenAI\genai-personal\.venv\Lib\site-packages\pydantic\main.py:253: UserWarning: WARNING! top_p is not default parameter.
    top_p was transferred to model_kwargs.
    Please confirm that top_p is what you intended.
validated_self = self.__pydantic_validator__.validate_python(data, self_instance=self)

```

Start coding or [generate](#) with AI.